ChatGPT

# Projection Mapping Auto-Calibration & Segmentation Plan

## Overview of the Approach

In this project, we aim to create a prototype that **automatically identifies projection surfaces and warps content onto them** using a phone camera and a projector. The core idea is to have the projector display known patterns (a white screen and a checkerboard), capture images from the projector's perspective, and then use computer vision – specifically Meta's Segment Anything Model (SAM) – to segment the scene into distinct surfaces. The selected surface segments will then be used to **automatically compute a mapping** so that projected content aligns correctly on those surfaces, accounting for perspective distortion and depth as much as possible. This approach minimizes manual mapping and leverages **cloud-based processing** for heavy tasks (image segmentation, pattern analysis) to keep the mobile/web client lightweight.

*If true 3D depth capture and perfectly precise warping prove too complex for now, the plan will still yield a very cool demo by focusing on planar or piecewise-planar mapping.* We will note potential challenges (camera alignment, multi-depth surfaces, etc.) and suggest reasonable workarounds. The following implementation plan is organized in phases, with clear steps to quickly start prototyping.

## Phase 1: Scene Capture & Calibration

**1.1 Setup and Camera Positioning:** Place the phone (or a camera) as close as possible to the projector's lens position, facing the projection area. The goal is for the camera to see nearly what the projector "sees" – this simplifies mapping since the perspective will be similar. In an ideal scenario, the camera and projector are fixed rigidly together (as done in some research setups [1]), but for a quick prototype you can manually hold the phone at the projector location. Ensure the environment can be darkened, since we'll rely on the projector's light for our images.

**1.2 Project a White Screen to Identify the Projection Area:** Start by projecting a full-white image through the projector (e.g., an all-white webpage or canvas at projector resolution). With the room lights dimmed, use the phone camera (in a web app or mobile app) to **capture a photo of the scene**. This image will show a brightly illuminated region where the projector's light falls, and darker elsewhere. From this image, detect the **edges of the projection** on the surfaces: - Use a simple threshold or edge-detection on the captured image to find the boundary between the bright projected area and darkness. This gives an outline (possibly an irregular shape) of the projection coverage. - This outline is important to **mask out anything outside the projector's view** and focus subsequent processing. It also effectively tells us the projector's field-of-view on the scene. - (If the projector's entire image lands on surfaces, the outline might be a distorted quadrilateral. If some corners go off into space with no surface, the outline will be cut off by the surface boundaries.)

**1.3 Project a Calibration Pattern (Checkerboard):** Next, project a known **checkerboard or grid pattern**. For example, a black-and-white checkerboard that fills the projector output. You might use a few patterns in sequence (e.g., checkboard squares getting smaller each time) to capture multiple scales of detail – but to start, one mid-sized checkerboard pattern is fine. For each pattern: - Capture another photo with the phone. The checkerboard will appear distorted on the surfaces. - Use computer vision (likely on the backend/cloud) to detect the pattern's keypoints. For example, use OpenCV's `findChessboardCorners` on the image to find the corners of the squares, or detect the grid intersections. Each detected point in the camera image corresponds to a known coordinate in the projector's output image. - **Map projector pixels to camera pixels:** From these correspondences, calculate a mapping. If the entire scene were a single flat plane, these points would yield a neat homography mapping from projector space to camera (and thus to the physical surface) [2]. In a multi-surface scenario, one homography won't perfectly model the whole scene due to depth differences, but it's still useful. We can either: - Compute a **global warp** that approximates the overall mapping (this could be a homography or a more complex distortion map). This won't be perfect if surfaces have different depth, but it provides a baseline. - Or simply **store the point correspondences** for later use. We might apply them per segmented surface in a later step. - *Note:* If pattern detection is noisy (e.g., due to shadows or surface texture), consider capturing multiple patterns (like shifting the checkerboard or using a colored grid) and combine the points. Also ensure the projector pattern is fully visible in the photo (adjust camera exposure if needed, since a bright projector can cause glare).

**1.4 Additional Depth Pattern (Optional):** For a more advanced prototype, you could project a series of structured light patterns to deduce depth. For example, sequential checkerboards or gray code patterns with progressively finer resolution (small squares) can capture more points on the surfaces. This is how professional systems achieve precise alignment – **structured light scanning provides a detailed 3D shape of the scene** [3]. However, implementing a full 3D reconstruction from patterns is complex. For now, we acknowledge this as a future enhancement. The current POC will proceed assuming either a roughly planar scene or using per-surface homographies for each segment.

**Potential Issues in Phase 1 & Solutions:** - *Camera not exactly at projector perspective:* Any offset means the captured images are from a slightly different angle than the projector. This can introduce mapping errors. **Mitigation:** Keep the phone lens as close to the projector lens as possible; use the calibration pattern to correct for small offsets. For example, the global homography computed from the checkerboard effectively compensates for a different viewpoint to the extent the scene is planar. If multiple depths, exact compensation is harder – but as long as the phone is within a few inches of the projector, errors will be small for distant surfaces. - *Ambient light or reflective surfaces:* Too much ambient light can reduce the contrast of the projected pattern, making it hard to detect. Highly reflective or transparent surfaces might not show the pattern clearly. **Mitigation:** Dim or turn off lights during calibration. Use a matte pattern (e.g., avoid too much white in the checkerboard that could cause glare – include some gray if needed to avoid overexposure). You can also take multiple photos and adjust camera settings (if using a custom app or some manual camera control). - *Projector focus:* If some surfaces are significantly farther than others, parts of the pattern might be out of focus, blurring the checkerboard. This could make corner detection difficult. **Mitigation:** Try to focus the projector for a middle distance, or use a pattern with thicker lines that still registers even if slightly blurred. In a POC, you might also constrain the scene to a single depth plane (like one wall) for simplicity.

## Phase 2: Automatic Segmentation of Surfaces (using SAM)

**2.1 Run SAM on the Captured Scene Image:** Take the photo from step 1.2 (the scene lit by the white projection) and feed it into the Segment Anything Model. This will likely be done on the backend (cloud server) since SAM is heavy. The SAM model can **generate segmentation masks for all prominent objects or regions in the image** without any custom training [4]. In practice: - Load the SAM model (e.g., the ViT-based default model) in a Python service. Input the scene image. - Use SAM's capability to get **all segmentation masks** above a certain confidence or area threshold. SAM can return dozens of masks – each mask is basically a binary region covering an object or part of an object. - Because we only care about areas within the projector's field (the bright region), **filter the masks** using the projection outline from step 1.2. For example, discard any mask that lies mostly outside the lit area. This ensures we focus on surfaces the projector actually hits. - What remains should be candidate surfaces on which to map content. For instance, if the projection covers a wall and a few objects, SAM might produce masks for the wall (maybe split into sections if there are color differences), for each object, etc.

**2.2 Review/Refine Segmentation (Optional):** Automated segmentation is not perfect. SAM might: - Over-segment (e.g., cut one physical surface into multiple masks if textures or colors vary). - Under-segment (e.g., group two adjacent surfaces as one mask if they have similar appearance). - Miss very small or subtle surfaces.

For a robust app, you might need a quick refinement step: - One approach is to allow **SAM interactive prompts**. The user (or an automated heuristic) can prompt SAM to merge or refine certain masks. For example, if the wall is broken into two masks, you could drop a point prompt in each to combine them, or simply let the user select both as one. - Another approach is to use depth or geometry if available (not in our basic POC) to inform segmentation (this is something researchers are exploring with SAM3D [5], but we'll skip that now).

For the prototype, we can likely proceed with SAM's output and rely on user selection to pick the right segments.

**2.3 User Interface for Segment Selection:** In the front-end app (TypeScript web interface): - Display the captured image (from 1.2) to the user. Overlay the SAM segmentation masks with distinct translucent colors or outlines. - The user can tap/click on the regions corresponding to surfaces they want to projection-map. For example, they might click on the mask covering a certain wall section, or a mask on a piece of furniture. - When a mask is selected, highlight it clearly (and possibly label it). - Allow selecting multiple masks if the user wants to map multiple surfaces in one go. - If SAM produced too many irrelevant masks, the user can ignore those – we only act on the selected ones.

**2.4 Capture Selected Mask Data:** Once selection is done, gather the mask information for those chosen surfaces. You'll have something like a binary mask or a polygon for each chosen region in image coordinates.

**Potential Issues in Phase 2 & Solutions:** - *Segmentation accuracy:* The SAM model might not perfectly align with the true edges of a surface (especially in low-contrast or very geometric scenes). **Mitigation:** Since we projected a white screen, the lighting is uniform, which helps reveal edges. If a surface is a solid color, SAM might have difficulty – but generally SAM is trained on a wide variety of images so it should detect edges of objects or surfaces. If a mask is slightly off, our projection content might spill a bit; for the POC this is

acceptable. For improvement, one could manually adjust the mask edges in the UI (out of scope for quick prototype). - *Too many segments (user confusion):* SAM could return many masks (dozens). **Mitigation:** We can filter by size (ignore very tiny masks) or by whether they lie in the projection area. We can also implement a simple rule: highlight only the largest few masks initially, assuming those are likely the main surfaces, and let the user reveal more if needed. - *Performance:* Running SAM on a high-res image might be slow. **Mitigation:** We can resize the image before segmentation (SAM is robust to resizing). Also, leveraging a GPU on the server is ideal. If a server GPU isn't available, consider using a smaller model like **MobileSAM**, a compact version of SAM optimized for speed on edge devices [6]. MobileSAM delivers near-instant segmentation results at the cost of some accuracy, and is compatible with the SAM pipeline if needed [6]. This could even run in-browser with TensorFlow.js or via WebAssembly if we go fully web, but initially a server approach is simpler.

## Phase 3: Surface Mapping & Warping Calibration

Now that we have the target surfaces (segments) selected, we need to figure out how to **project content onto them** such that it fits their shape and position.

**3.1 Mapping Coordinates via Calibration Data:** If the calibration checkerboard was used (from step 1.3), we have correspondences between projector coordinates and camera image coordinates across the scene. We can use this to map content: - For each selected segment mask, filter the set of checkerboard points to those that fall inside that mask region (i.e., points that were projected onto that surface). - If the surface is approximately planar and we have at least 4 correspondences on it, compute a **homography** between projector image space and that camera image region. This homography essentially tells us how to warp any content from the flat projector's perspective into the shape of that surface as seen by the camera [2] [7]. In other words, it accounts for that surface's orientation and distance. - If a surface had very few checkerboard points (e.g., a small object that only showed a couple of squares), the homography might be unreliable. In that case, you could fall back to using the global approximation or even manual adjustment if needed.

If we did *not* do the pattern calibration, or want a simpler route for the POC: - We can assume the **camera's view is the projector's view**. Then the segmentation mask itself (in camera image coordinates) can serve directly as the projection mask. Essentially, if we project an image that has the same mask shape cut out, it should land on that physical surface (because the camera and projector views are nearly the same). - This is a big assumption, but for a quick demo it often works: for example, researchers sometimes just align a camera with a projector and treat them interchangeably [8]. By observing a known pattern's projection in the camera, one can infer the scene structure and alignment [9]. We are taking a shortcut by using the mask silhouette itself as the target shape for projection.

**3.2 Generating Warped Content:** Depending on what the user wants to project, there are two approaches: - **Option A: Simple Highlight/Color Projection:** Easiest for a POC – just project a solid color or a simple texture onto the chosen surfaces to prove the mapping works. For each segment, create an image (same resolution as projector) that is black everywhere except that segment's area, which is filled with, say, white or a test pattern. This image, when sent to the projector, will light up exactly that surface region. You can do this by taking the binary mask of the segment (in camera coordinates, which we assume correspond to projector coordinates for now) and pasting it onto a black background of projector resolution. - **Option B: Warping an Arbitrary Content Image:** For a cooler demo, allow the user to provide an image or use a sample graphic to map onto the surface. If we have a homography for the surface (from calibration): - Use OpenCV or a similar library to warp the input image through the homography **from the surface's plane**

**back to the projector's plane**. Essentially, if the user has an image that is shaped like the physical surface (for instance, a rectangle design for a wall section), the homography transform will skew it into the trapezoidal shape that the projector needs to output so it appears correctly on that angled surface. - If we are not using calibration data, another method is to take the bounding box of the segment in the camera image, map the user's content into that box, and then multiply by the mask (so it only shows on the irregular shape). This won't correct perspective perfectly, but it will fill the shape. For example, you could simply texture-map the content onto the mask region via a canvas 2D context. - Do this for each selected surface. If multiple surfaces are selected, you might generate separate projector images for each or combine them into one if you want to project on all at once (just ensure they don't overlap in projector view).

**3.3 Accounting for Depth and Warping:** In an ideal scenario, we would have actual depth information for each surface to perfectly account for foreshortening and size. In our POC: - Using the homography per surface already accounts for the first-order perspective distortion if the surface is planar. That means the content will **appear undistorted when viewed from the projector's angle**, fitting the surface's shape. (It may still look skewed to an observer from a different angle, but that's expected in projection mapping.) - If surfaces are not planar or have significant relief, this method will struggle. The closest we can come without deeper 3D mapping is to subdivide such surfaces into smaller planar patches and handle each (which is complex), or simply accept some distortion. - It's worth noting that research projects (like SAM3D and others) are looking into combining segmentation with depth sensing [5]. In a future iteration, one could incorporate a depth camera or use multiple images to reconstruct a point cloud, then **map the 2D masks into a 3D model** [5]. That would enable truly accurate projection on arbitrary shapes. For now, our approach will stick to approximate but visually effective warping with homographies.

**Potential Issues in Phase 3 & Solutions:** - *Alignment errors:* If the camera and projector views differ or if the homography is off, the projection might spill over edges or not fully cover the surface. **Mitigation:** Provide a way to iterate – after projecting, the user can observe and perhaps go back and adjust. In a dev setting, you could even overlay the projected result in the camera feed to see alignment. Minor tweaks (like nudging control points of the mask) could then be made and re-tested. - *Multiple surfaces with one projector image:* If you combine several surfaces into one projection output, ensure that their content doesn't overlap in projector space. Usually if the surfaces are separate in the physical world, their masks won't overlap in the camera/projector image either. If they *do* overlap (maybe one object in front of another), SAM segmentation likely would have separated them correctly. Just be mindful to composite the final output image such that you don't double-write to the same pixel. - *Intensity fall-off:* Surfaces farther from the projector will receive less light (appearing dimmer) and at oblique angles the brightness per area is lower. Our system won't explicitly correct brightness or focus, but as an improvement you could compensate by increasing brightness for distant/angled surfaces if noticed. - *Latency:* The process from capturing images to sending to cloud and back to projecting content might take some time (several seconds). For a proof-of-concept, this is fine. Just ensure the user knows to hold the camera steady until capture is done, and maybe freeze the projector output while processing.

## Phase 4: Testing the Prototype and Iteration

**4.1 Integrated Test Workflow:** Now, bring it all together: - The user (or developer) sets up the projector facing a scene and goes to the web app (or runs the script). - Step by step, have the projector display the white screen, prompt the user to take a photo (this could be automated in the app by showing the white and capturing from camera stream). - Then display the checkerboard pattern, capture that photo. - Send the

images to the server for processing (segmentation and optionally calibration). - Receive the segmentation masks, display to user for selection in the browser. - User selects the surfaces they want. Perhaps selects a content option (color or an image) for each. - The app generates (or requests from server) the final warped projection image. - Projector displays the mapped content onto the scene.

Every step can be semi-automated with prompts, but for a quick prototype it might be a manual step-through (e.g., a "Capture White Image" button, then "Capture Pattern" button, etc.).

**4.2 Evaluate Result on Real Surfaces:** With the content now projected: - Observe if it indeed lands only on the chosen surfaces and aligns with their boundaries. This is the moment of truth for the POC. - If something is misaligned, note where the process might be improved: - If edges are off, maybe the segmentation mask was a bit off or the homography was not perfect – could adjust mask or add more calibration points. - If the whole image is shifted, maybe the camera moved slightly after calibration – ensure the phone/camera doesn't move between capturing and projection, or redo calibration quickly if it did. - Try different content or surfaces to demonstrate versatility: for instance, select another object in the scene and project onto it.

**4.3 Iterate on Problem Areas:** If a particular issue arises frequently, plan a fix: - For example, if the segmentation splits a single surface, you might add a step to merge masks (perhaps by detecting adjacent masks on the same plane via calibration points). - If calibration is the weak link (e.g., if you skipped it and alignment is poor), you might implement at least a simple 4-point homography using the corners of the projection area outline: map the quad outline in the camera image to the projector's rectangle. This at least ensures the overall image is aligned [2]. Then within that, the segmentation mask will be roughly in the correct place. - In a longer-term project, consider using AR libraries (ARCore/ARKit) to get approximate surface planes and pose – but since we want web-based and general, we stuck to our custom approach.

At the end of this phase, you should have a working proof-of-concept that **automatically scans a scene and projection-maps content onto chosen parts**. It may not be pixel-perfect on complex geometry, but it demonstrates the concept of rapid, automated projection mapping.

## Implementation Details & Tech Stack

**Frontend (Web, TypeScript):** Use a lightweight web app to handle user interaction and device capabilities: - **Framework:** You can use plain TypeScript with a minimal bundler, or a simple React/Vue app if you prefer structure. Next.js could be used especially if you want server-side functions, but a small single-page app might suffice. - **Camera Access:** Use the Web Media API (`navigator.mediaDevices.getUserMedia`) to access the phone's camera. You can embed a video preview for alignment, and capture a frame (canvas `.captureStream()` or draw ImageBitmap to canvas and getImageData) when needed. Ensure the resolution is high enough for segmentation (720p or above). - **Projector Output:** If using the same device for projection, you might open a new window or use the Screen Capture API to present the final output. However, on mobile, using the same phone to project is uncommon unless it's a specialized device. More likely, you will use a laptop with a projector for actual testing. In that case, the web app can be opened on the laptop and the laptop's webcam used as the camera (placed near the projector). This might be the easiest test setup. - **UI Workflow:** Implement buttons or instructions for each step: e.g., *"Click to show white screen and capture"*, then *"Show pattern and capture"*, etc. You can have the projector display be just a full-screen div that changes background (white, then pattern). After capture, switch that div off or to black so it doesn't interfere. - **Displaying Masks:** After segmentation results come back, draw the masks on a canvas

or use SVG overlays. Make them semi-transparent colored regions over the photo. Add event listeners so clicking a region selects it. (Since SAM may give masks as pixel-wise bitmaps or perhaps polygons, you may need to convert them to a format for display – drawing the binary mask onto a canvas with some color is straightforward.)

**Backend (Cloud or Local Server):** This will handle heavy computations: - **Technology:** Python is recommended (with Flask/FastAPI for an API) because of available libraries (PyTorch for SAM, OpenCV for image processing). Alternatively, a Node.js server could delegate to Python scripts or use TensorFlow.js, but that's more effort. - **Segment Anything Model:** Use Meta's official SAM model (available via PyTorch and ONNX). Load the model weights (which are a few hundred MB) on the server start. Provide an API endpoint like `/segment` where the server accepts an image (from the white projection capture) and returns mask data. Mask data can be encoded as PNGs or as JSON polygons. Given bandwidth, you might simplify by returning a downscaled mask image overlay. - **Pattern Calibration:** Implement a `/calibrate` endpoint. When the checkerboard capture is sent, use OpenCV's calibration functions: - `findChessboardCorners` to get image points. - Since you know the checkerboard pattern (e.g., 8x8 grid of squares and square size), you can also define the object points in 3D or in projector 2D coordinates (if planar projection, treat it as z=0 plane in projector space). - Use `findHomography` or even `calibrateCamera` if you want full intrinsics. But a direct homography from projector 2D to camera 2D might suffice if assuming planar. - Return either the homography matrix or the set of point pairs. - **Warping Content:** You can also have the server do the final warping/compositing: - For instance, a `/warp` endpoint that takes an image (content) and a homography (or mask) and returns a warped image ready for projection. - However, doing this on the frontend can save a round trip. It's just a matter of convenience vs. complexity. For POC, doing it in Python with OpenCV (using `warpPerspective`) is straightforward and ensures correctness. - **Cloud vs Local:** If you deploy this, ensure the images (which might be a few MB) can be uploaded quickly. Locally, this is not an issue. If using cloud, some latency in segmentation is okay as this is an offline calibration step, not a continuous real-time process.

**Data Flow:** 1. Frontend captures images (white, pattern) → send to server. 2. Server processes and returns segmentation masks (and maybe calibration info). 3. Frontend displays masks → user selects surfaces → send selection (and maybe content) to server or handle locally. 4. Server (or frontend) produces final projection image → frontend displays it on projector.

**Cleanup and Reuse:** - Make sure to handle any memory issues (e.g., releasing large images, model only loaded once). - The prototype can be run repeatedly, so you might keep the calibration data in memory so if the user reselects surfaces or content you don't need to recalibrate unless the setup changed.

## Anticipated Challenges & Workarounds

- **Precise Geometry Mapping:** Truly accounting for 3D shape with just a few images is challenging. Our approach approximates it. The closest current tech (without specialized depth cameras) is structured light scanning, which *Lightform* devices used to achieve precise alignment [3]. If our simpler method isn't sufficient, consider integrating more patterns or even using the phone's AR depth sensing (if available) as a fallback to get a point cloud of the scene.
- **Web Limitations:** Running heavy CV in a browser is not practical for now – that's why we offloaded to cloud. This requires internet connectivity. If a fully offline solution is needed, the mobile app

version could embed these models (bearing in mind size and performance). As noted, **MobileSAM** or other efficient models could be leveraged for on-device segmentation in the future [6].

- **User Experience:** We must ensure the process is understandable: guiding the user when to point the camera and when the projector will flash patterns. It might be confusing to do alone, so clear on-screen instructions are key ("Aim your phone at the projection and press 'Capture'"). Also, after the auto-mapping, the user should see the result quickly to build confidence.
- **Accuracy vs. Speed:** There's a trade-off between how fine the mapping is and how quick the process runs. For a flashy demo, you can err on the side of speed (maybe use just one pattern image and a coarse alignment). If demonstrating technical capability, you might take a bit more time to ensure the projection fits perfectly (even manually tweaking masks if needed).
- **Fallback if Not Possible:** If we find fully automatic mapping of a complex scene isn't working with current tech, the fallback "closest cool version" would be:
- Assume a **mostly planar scene** (like a single wall or flat projection surface with some decals). The calibration then reduces to a single homography which we can do reliably [2].
- Use SAM to find sub-areas on that plane (e.g., posters, windows on a wall) so the user can select one to highlight. This is still very cool: you'd point a projector at a wall with various things on it, and the app can automatically detect and let you illuminate one object (say, light up a painting on the wall) with one click.
- This simplifies depth issues and could be achieved with high confidence. Essentially it becomes *"point projector & camera at wall, it finds objects on the wall, and maps content onto one automatically."*

By following this implementation plan, you can rapidly prototype the projection mapping app extension. The plan accounts for key issues (calibration, segmentation, warping) and provides alternatives if certain ideal steps aren't feasible. The end result will demonstrate the power of combining structured light ideas with advanced segmentation – creating a seamless augmented reality projection that maps onto real-world surfaces with minimal user effort. Good luck, and have fun seeing your **projector intelligently paint only the objects you choose**! [4] [9]

---


[1] [2] [7] [8] [9] intellar.ca - Augmented reality with camera projector mapping
https://www.intellar.ca/blog/augmented-reality-with-camera-projector-mapping

[3] Lightform – Design Tools for Projection. Lightform makes it easy for anyone to create epic visuals for projected AR using content creation software powered by computer vision hardware.
https://lightform.com/

[4] Master Image Segmentation with SAM's Zero-Shot AI
https://viso.ai/deep-learning/segment-anything-model-sam-explained/

[5] GitHub - Pointcept/SegmentAnything3D: [ICCV'23 Workshop] SAM3D: Segment Anything in 3D Scenes
https://github.com/Pointcept/SegmentAnything3D

[6] Mobile Segment Anything (MobileSAM) - Ultralytics YOLO Docs
https://docs.ultralytics.com/models/mobile-sam/